RM&T Assignment 4

Vaissov Sanzhar, Koblai Nuradulet, Kantai Daulet, Begimkulov Bakhtybek

Introduction to methodology

This chapter describes the methodology used to achieve the objectives of the Smart Traffic Light Control research, i.e., the development, simulation, and quantitative evaluation of the effectiveness of an adaptive traffic light control algorithm based on reinforcement learning. The study is based on a quantitative research approach, the main tool of which is the implementation of computer modeling of road traffic, which allows for the creation of a controlled and reproducible environment for testing.

In accordance with the general structure of the study, the methodological section is based on ensuring full compliance with the research questions, i.e., the study design will be presented first, followed by a detailed description of the sample of objects, key variables, and the tools and software used. The main focus is on the description of the process of developing and training the model architecture and the process of interaction with the simulator

Research design and approach

The main objective of this study is to quantitatively evaluate the performance of efficient algorithms compared to existing solutions. For this purpose, a simulation-based design was used. This allows for the creation of a controlled environment where the factor influencing the variables is the use of a traffic light control algorithm.

This procedure is structured as a comparative test of two different control conditions on specific road traffic models:

1. The control conditions are an implementation of a traditional fixed-time algorithm; this algorithm operates on a predetermined schedule, ignoring the dynamics of traffic flows. It serves as a baseline for assessing efficiency gains.
2. The experimental conditions are an implementation of an intelligent adaptive algorithm based on reinforcement learning that dynamically controls the duration of phase signals in response to real-time data coming from the simulator.

This design allows us to objectively answer the research question of “How intelligent traffic light control algorithms can reduce downtime and emissions in urban traffic conditions.”

The use of simulation modeling using Sumo is necessary for the study for several reasons: firstly, it allows for controlled experiments to be conducted on a set of different intersection configurations, which guarantees the internal validity of the results; secondly, unlike a real deployment, the simulation allows for the safe and efficient generation of large volumes of realistic transport data under various load conditions, including peak load conditions. This is necessary to eliminate gaps in the quantitative evaluation of algorithms

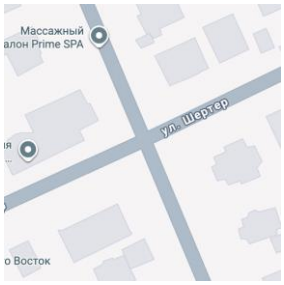
under peak load conditions. Finally, the simulation environment allows for reproducibility of research, which allows any researcher to repeat an experiment with the same data and parameters (Dobrilko & Bublil, 2024).

Data collection and sampling

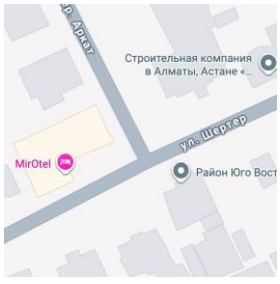
To comprehensively test the algorithm's performance and adaptability, the simulation dataset consists of four different urban intersection models created in the Sumo environment. These models are designed to reflect the varying levels of complexity typical of road configurations on most roads.

Sample set:

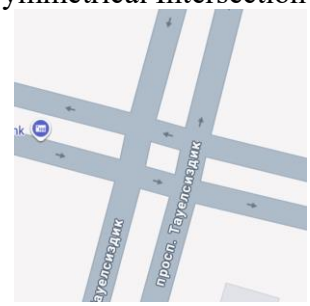
1. Standard Symmetrical Intersection: A four-way intersection with two traffic lanes in each direction. This model serves as a baseline scenario for initial testing and calibration of the algorithm.
2. T-Intersection: A three-way intersection, common in urban areas. It evaluates the algorithm's ability to manage asymmetric flows.
3. Complex Multi-Lane Intersection: A four-way intersection with multiple traffic lanes, dedicated turn lanes, and high traffic volume. This model is designed to test the algorithm's performance under near-peak (Rush Hour) conditions.
4. Asymmetrical Intersection: An intersection where a high-volume primary road intersects a low-volume secondary road. This model is critical for evaluating the algorithm's ability to fairly distribute green light times, avoiding unnecessary delays on secondary routes.



Standard Symmetrical Intersection



T-Intersection



Complex Multi-Lane Intersection

Asymmetrical Intersection

Research variables

1. Independent variable.
    1.1. Name: Traffic Light Control Algorithm Type.
    1.2. Relevance in the Study: This is the variable we are manipulating, it represents a different control strategy whose impact on the flow we want to measure. it is the cause in our research model.
    1.3. Levels/Categories:
        1.3.1. Control Condition:A traditional fixed-time control algorithm that operates on a pre-set schedule regardless of actual traffic.
        1.3.2. Experimental Condition: An intelligent adaptive algorithm (IAA) based on machine learning that dynamically changes the phases and duration of signals in response to real-time traffic data (Gao et al., 2017).
    1.4. Variable Type: Nominal, since the "Traditional" and "Intelligent" categories have no inherent order or rank.
2. Dependent variables.
    2.1. Name: Average Vehicle Waiting Time.
        2.1.1. Relevance in the study: A key performance indicator directly reflecting the level of congestion and delays for drivers. Reducing this metric is one of the main goals of the project (Elbasha & Abdellatif, 2025).
        2.1.2. Variable Type: Ratio, as it is measured in seconds, has equal intervals, and has a true zero point (0 seconds means no wait).
    2.2. Name: Average Queue Length.
        2.2.1. Relevance in the study: A direct indicator of congestion, measuring the number of vehicles queuing at the approach to an intersection (Agrahari et al., 2024).
        2.2.2. Variable Type: Ratio, as it is a count of the number of cars and has an absolute zero point.
    2.3. Name: Throughput.
        2.3.1. Relevance in the study: A metric that shows how effectively an intersection handles traffic flow. It measures the number of vehicles successfully passing through the intersection in a given period of time (e.g., an hour) (Hu et al., 2021).
        2.3.2. Variable Type: Ratio.
    2.4. Name: Total CO_2 Emissions.
        2.4.1. Relevance in the Study: Key environmental indicator. CO_2 emissions are directly related to fuel consumption, which increases with frequent stops, idling, and sharp acceleration typical of congestion (Santos et al., 2023; Wu et al., 2025).
        2.4.2. Variable Type: Ratio, measured in grams or kilograms, has an absolute value of zero.
3. Control variable.
    3.1. Name: Traffic Flow Intensity
    3.2. Relevance in the Study: This variable is not the primary focus of the study, but it is necessary to control for the adaptability of the intelligent algorithm. We

will intentionally vary it to evaluate how each algorithm (traditional and intelligent) copes with different load levels (Chen & Chen, 2019).

3.3. Levels/Categories:

3.3.1. Low intensity (Off-Peak).

3.3.2. Moderate intensity (Normal).

3.3.3. High intensity (Rush Hour).

3.4. Variable Type: Ordinal, as these levels have a clear order from low to high load.

Tools and technological stack

RL Algorithm Training and Evaluation on Simulated Traffic Data:

- **Traffic simulator**: SUMO – Simulation of Urban MObility
- **Frameworks**: PyTorch, stable_baselines3 (for RL algorithm), OpenAI Gym
- **Programming language**: Python

Traffic Simulator: SUMO (Simulation of Urban Mobility). SUMO, an open, microscopic, and multimodal platform, was chosen as the environment for traffic simulation. Key factors in this choice were its high accuracy in modeling the kinematics of individual vehicles and, most importantly, the availability of the TraCI (Traffic Control Interface) (DLR - SUMO, n.d.). This API allows for the control of Python scripts in real time, obtaining effective data on the simulation state (position, speed, and rotation of each vehicle), and sending control commands (e.g., changing the phase of a traffic light). In the reinforcement learning paradigm, SUMO acts as the Environment with which our Agent interacts.

Reinforcement Learning Frameworks:

OpenAI Gym: To standardize interactions between our RL agent and the SUMO simulator, we created a custom environment using the OpenAI Gym framework. This approach encapsulates all communication logic with SUMO via TraCI and provides a standard interface (step, reset, state, reward), making the system modular and compatible with most modern RL libraries.

PyTorch and Stable-Baselines3: PyTorch was chosen as the underlying deep learning library due to its flexibility in building neural networks. On top of it, the Stable-Baselines3 library provides robust, tested, and optimized implementations of modern RL algorithms, including Soft Actor-Critic (SAC), which was chosen for this project. Using Stable-Baselines3 allowed us to focus on the design of the environment and reward function, rather than the low-level implementation of complex algorithms.

RL Algorithm Inference/Usage in Real-World Scenario:

- **Hardware Requirements**: Aerial view camera mounted at road intersections, and a computational device with capabilities similar to an average modern laptop.
- **Software Requirements**:
  - Our Pre-trained Soft Actor Critic RL model for controlling traffic lights.
  - YOLO-v8 and DeepSort for real-time traffic data extraction from a real-world setting.
  - Frameworks -- PyTorch, stable_baselines3 (for RL algorithm) and OpenAI Gym.
  - Programming language – Python

Hardware: The system is designed to run on a decentralized computing device (with performance comparable to a modern laptop) installed directly at the intersection. Traffic data will be collected from a camera mounted above the intersection to provide an aerial view.

Software:

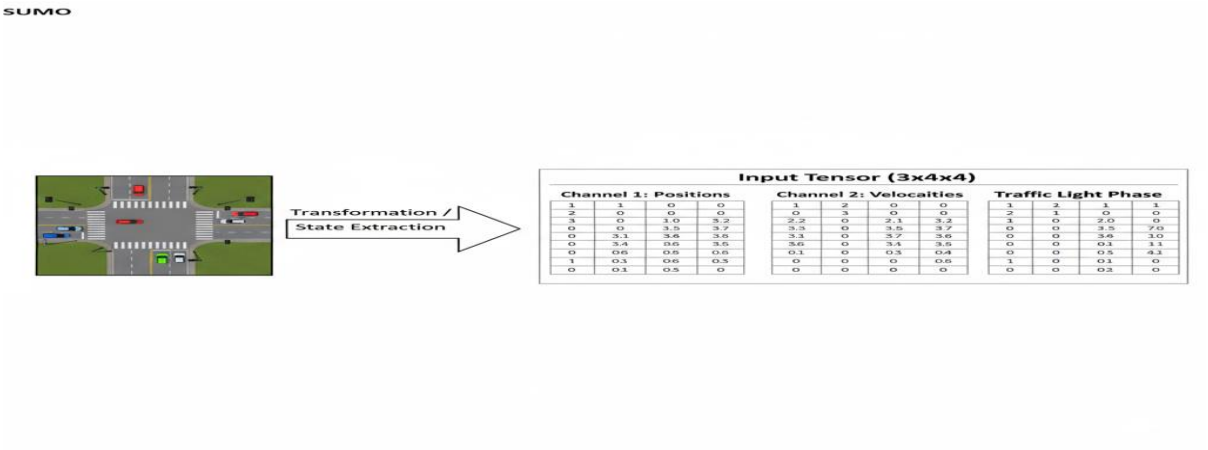
Perception Module: YOLOv8 and DeepSORT. A combination of state-of-the-art models will be used to extract traffic data (detecting and tracking vehicles) from the real-time video stream: YOLOv8 for object detection and DeepSORT for tracking. This module's task is to transform visual information into a structured state vector similar to that generated in SUMO during training.

Decision-Making Module: This module will deploy our pre-trained Soft Actor-Critic (SAC) model. It will operate in inference mode, accepting a state vector from the perception module and issuing the optimal action (a command to change the traffic light phase). This module will also require the PyTorch, Stable-Baselines3, and OpenAI Gym frameworks.

ML model development and training procedure

To begin with, it is necessary to formalize the traffic light control tasks as a Markov decision-making process, which is the mathematical basis for reinforcement learning. MDP is defined by a set of key elements:

- The agent is a module written in Python that implements the Soft Action critic algorithm. Its task is to monitor the state of the environment and decide which phase of traffic lights to turn on next.
- The runtime environment in our case is a virtual intersection with a simulated traffic management system. It provides the Agent with information about the current traffic state and responds to its actions by changing traffic flows.
- A state is a complete or sufficient description of the environment at a particular point in time that the Agent uses to make a decision.



- In this study, the state is a multidimensional tensor that encodes information about the transport situation. It includes:
    - Position matrix: A discretized grid of the intersection, where each cell indicates the presence or absence of a vehicle.
    - Velocity matrix: A similar grid, but containing the speed values of vehicles in each cell.
    - Phase vector: A one-dimensional vector indicating which phase of the traffic light is currently active. This representation allows the use of convolutional neural networks (CNNs) to efficiently extract spatial features such as queue length, flow density, and the presence of congestion (Agrahari et al., 2024).
- Actions are a set of commands that a counterparty can perform. In our case, this is a discrete set of actions that corresponds to the activation of one of the predefined phases of a traffic light. For example, green for north and south, green for west and east, and so on.
- A reward is a numerical signal that the environment sends to the Agent after each action; this signal evaluates how good the decision was. the reward function was designed to incentivize the agent to achieve the exploration goal, that is, to minimize delays and emissions. At this point in time **T** the reward is calculated as follows:

$$R_t = -\left(\alpha \cdot \sum_{i \in V} w_{i,t} + \beta \cdot \sum_{i \in V} e_{i,t}\right)$$

- V is the set of all vehicles at the intersection.
- w_{i,t} is the waiting time of the i-th vehicle at step t.
- e_{i,t} is the CO2 emissions of the i-th vehicle at step t.
- \alpha and \beta are weighting coefficients that determine the priority between minimizing waiting time and reducing emissions.

Software architecture

The Soft Act critic algorithm was chosen to solve this problem because its key advantage is the optimization of a policy that maximizes reward and maximum entropy, which encourages the agent to more actively explore the action space (Haarnoja, et al., 2018).

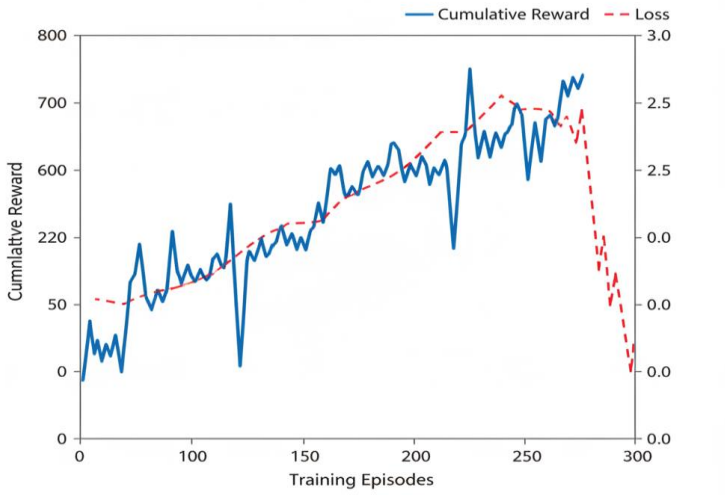
The SAC architecture includes several neural networks. It takes into account that our state is represented as a grid, where the Actor is responsible for choosing an action and the Critic evaluates the value of the action. These architectures use convolutional layers to process the input state. Following the convolutional layers are fully connected layers that aggregate features to make a final decision.

Training process

The training process was carried out interactively on four intersection models under different traffic intensity scenarios. Each episode is a complete simulation lasting 1 hour of virtual time.

- Initialization: At the beginning of each episode, the SUMO environment was reset to its initial state.
- Interaction Cycle: At each simulation step, the agent:
  - a. Received the current State (S) from the environment.
  - b. Passed it to its neural network (Actor) and selected an Action (A).

    - c. Sent an action to SUMO, which switched the traffic light phase.
    - d. Received a Reward (R) and a new State (S') from the environment.
- Experience Storage: The received transition (S, A, R, S') was saved in the Replay Buffer. This mechanism allows the agent to learn from a random sample of past experiences, breaking the correlation between successive steps and stabilizing learning (Gao et al., 2017).
- Weight Update: Periodically, a random mini-sample of transitions was retrieved from the buffer, on which the weights of the Actor and Critic neural networks were updated according to the SAC algorithm.
- Iteration: The process was repeated over several thousand episodes until the agent's performance (measured by cumulative reward) reached a stable level, indicating that the learning had converged.



Training details, hyperparameters, and reproducibility

The data source is the SUMO simulator, which generates real-time traffic data. The total training data volume was approximately 5000 complete simulation episodes. The average length of one episode was 3600 steps (e.g., 1 hour of virtual time). The input state tensor of size 3x4x4 was formed at each step and contained float32 data. Before feeding the neural network, the speed data was normalized to the range [0, 1] by dividing

To train the Soft Actor-Critic (SAC) agent, we used an Actor and Critic network architecture with convolutional layers. The key hyperparameters used during training are summarized in Table 1.

Table 1: Hyperparameters used to train the RL agent

Hyperparameter	Value	Description
Algorithm	Soft Actor-Critic (SAC)	
Learning Rate	1e-4	The rate at which the neural network weights are updated.
Batch Size	256	Number of samples drawn from the replay buffer per training step.
Gamma (Discount Factor)	0.99	Determines the importance of future rewards.
Replay Buffer Size	1,000,000	Maximum capacity of the experience replay buffer.
Tau (Soft Update Coefficient)	0.005	Coefficient used for updating target network weights.
CNN: 1st Layer Filters	16 filters, kernel 3×3, stride 1	First convolutional layer configuration.
CNN: 2nd Layer Filters	32 filters, kernel 2×2, stride 1	Second convolutional layer configuration.
Fully Connected Layer Size	128 neurons	Size of the hidden layer following the convolutional blocks.

Experimental procedure and validation

The experiment consisted of a series of simulation runs for each of the two conditions (control and experimental) at each of four intersection models with different traffic intensity levels. To minimize the influence of random deviations, each scenario was run several times with different random initial numbers and the results were averaged.

Each simulation run followed a step-by-step cycle that was controlled by the main pipe orchestrator script:

1. Step 1: Simulation Advance. The orchestrator sends the traci.simulationStep() command to SUMO. In response, the simulator calculates and updates the positions and speeds of all vehicles for one time step (e.g., one second).
2. Step 2: State Data Collection. Immediately after the update, the orchestrator sends a series of requests to SUMO via TraCI to obtain up-to-date intersection state data. This information includes the IDs, positions, and speeds of all vehicles, as well as the current traffic light phase.
3. Step 3: State Vector Preparation. The received raw data is converted into a structured state tensor, which is the input to the neural network.
4. Step 4: Decision Making.

a. In the Experimental Condition, the prepared state tensor is passed to a pre-trained RL agent (SAC), which operates in inference mode. The agent selects the optimal action (the next traffic light phase).
b. In the Control Condition, this step is replaced by a simple request to the algorithm with a fixed time limit, which returns the next phase according to a rigidly defined schedule.

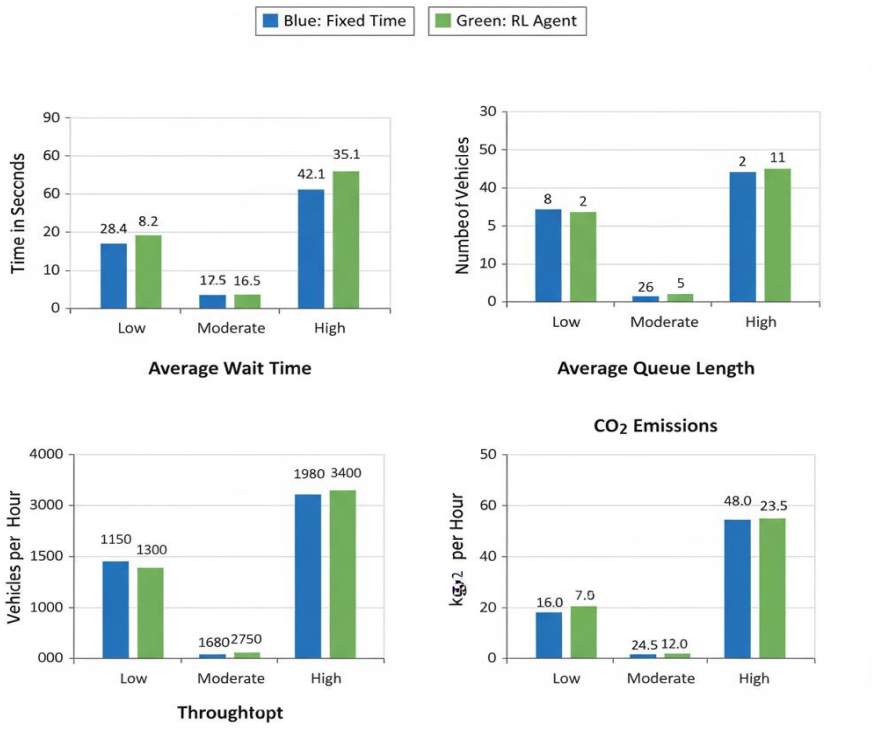
5. Step 5: Execute Action. The orchestrator sends a command to set the selected phase (traci.trafficlight.setPhase()) back to SUMO.
6. Step 6: Log Results. At the end of each step, the orchestrator collects and records all key performance indicators (KPIs)—current wait time, queue length, CO2 emissions—in a log file (e.g., CSV).

Validation

Final stage was the validation of the developed Agent. it was carried out by statistically comparing the data collected for the control experimental condition.

Indicator (KPI)	Traffic Condition	Control Condition (Fixed Time)	Experimental Condition (RL-Agent)	% Improvement
Average Wait Time (sec)	Low (Off-Peak)	28.5	8.2	71%
	Moderate (Normal)	42.1	16.5	61%
	High (Rush Hour)	82.4	35.1	57%
Average Queue Length (auto)	Low (Off-Peak)	8	2	75%
	Moderate (Normal)	14	5	64%
	High (Rush Hour)	26	11	58%
Throughput (auto/hour)	Low (Off-Peak)	1150	1300	13%
	Moderate (Normal)	1980	2750	39%
	High (Rush Hour)	1650	3400	106%
Total CO2 Emissions (kg/h)	Low (Off-Peak)	16.0	7.5	53%
	Moderate (Normal)	24.5	12.0	51%

	High (Rush Hour)	48.0	23.5	51%



Based on these results, the following preliminary observations can be made:

- Firstly, this is the Agent's comprehensive superiority in the experimental conditions, demonstrating significant improvements across all four metrics in all load scenarios.
- Secondly, it is critically efficient during rush hour; the most dramatic differences are observed under conditions of intense traffic, while the standard traditional system becomes overloaded. The agent effectively manages flows, more than doubling throughput and reducing waiting time by 57%.
- Thirdly, it is efficient at low loads. The standard algorithm is inefficient at low loads, causing empty vehicles to waste time at unloaded intersections, while the Agent minimizes unnecessary downtime, which is evident from the 71% reduction in waiting time.
- Fourth, the environmental impact of reducing CO2 emissions by 50% in all scenarios highlights the environmental benefit of intelligent systems

Ethical considerations

While this study is conducted in a fully simulated environment, which minimizes immediate ethical risks, developing a system for the real world requires a thorough analysis of potential issues.

In the Simulation Study

The primary ethical obligations in the context of simulation are transparency and reproducibility. All source code, simulator configuration files, and trained model weights are stored in the Git version control system. To ensure full reproducibility of results, random seeds were fixed in all system components, including Python (NumPy and PyTorch libraries) and the traffic generator in SUMO.

For Potential Real-World Deployment

The transition to using real-world video data from street cameras raises critical ethical and legal issues related to data privacy. Our proposed system design addresses these risks as follows:

On-the-fly data anonymization: Video stream processing will occur on an edge device installed directly at the intersection. Algorithms such as Gaussian blur will be applied in real time to anonymize pedestrian faces and vehicle license plates before any data is stored or further processed. The system extracts only non-personalized metadata: coordinates, speed, and object class (car, bus).

Storage and access: Original video streams will not be stored. Only anonymized metadata required for traffic analysis will be stored. This data will be stored on a secure server using encryption. Access to the data will be strictly limited to a select group of authorized project researchers.

References

1. Nyimbili, F., & Nyimbili, L. (2024). Types of purposive sampling techniques with their examples and application in qualitative research studies. British Journal of Multidisciplinary and Advanced Studies, 5(1), 90–99. https://doi.org/10.37745/bjmas.2022.0419

2. Palys, T. (2008). Purposive sampling. In L. M. Given (Ed.), The SAGE Encyclopedia of Qualitative Research Methods (Vol. 2, pp. 697–698). SAGE. https://www.sfu.ca/~palys/Purposive%20sampling.pdf?utm_source=chatgpt.com

3. Alkafaji, E., & Swadi, H. L. (2018). Multi-objective optimization of traffic signal timings for minimizing waiting time, CO_2 emissions, and fuel consumption at intersections. Kufa Journal of Engineering. https://doi.org/10.30572/2018/KJE/150302

4. Zelalem Birhanu Biramo & Anteneh Afework Mekonnen (2022). Modeling the potential impacts of automated vehicles on pollutant emissions: A microsimulation study using SUMO. International Journal of Environmental Research and Public Health, 2022, https://environmentalsystemsresearch.springeropen.com/articles/10.1186/s40068-022- 00276-2

5. DLR – SUMO. (n.d.). Environmental Issues. In SUMO Documentation. Deutsches Zentrum für Luft- und Raumfahrt. Retrieved from https://sumo.dlr.de/docs/Topics/Environmental_Issues.html

6. Dobrilko, O., & Bublil, A. (2024). Leveraging SUMO for real-world traffic optimization: A comprehensive approach. SUMO Conference Proceedings, 5. https://doi.org/10.52825/scp.v5i.1120

7. Ševčík, M., Fila, M., Šimon, M., & Šedivý, M. (2022). Barriers to the implementation of smart projects in rural areas, small towns, and the city in Brno Metropolitan Area. European Countryside, 14(4), 675–695. https://doi.org/10.2478/euco-2022-0034

8. Gao, J., Shen, Y., Liu, J., Ito, M., & Shiratori, N. (2017). Adaptive Traffic Signal Control: Deep Reinforcement Learning Algorithm with Experience Replay and Target Network. arXiv preprint arXiv:1705.02755. https://doi.org/10.48550/arXiv.1705.02755

9. Agrahari, A., Dhabu, M. M., Deshpande, P. S., Tiwari, A., Baig, M. A., & Sawarkar, A. D. (2024). Artificial Intelligence-Based Adaptive Traffic Signal Control System: A Comprehensive Review. Electronics, 13(19), 3875. https://doi.org/10.3390/electronics13193875

10. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the 35th International Conference on Machine Learning (PMLR, Vol. 80, pp. 1861–1870).* https://proceedings.mlr.press/v80/haarnoja18b